

int 0x30

SYS_EXEC | SYS_WAIT

SYS_READ | SYS_WRITE

SYS_OPEN | SYS_CLOSE

Relatorio de Deciso^{es} de Projeto

Projeto 2 — System Calls

Centro de Informatica — Universidade Federal de Pernambuco

Disciplina: Sistemas Operacionais

2025 / 2026

1. Sumario Executivo

Este relatório documenta as decisões de projeto da implementação do **Problema 2-2 do PintOS (Projeto 2 — Userprog)**: a infraestrutura completa de chamadas de sistema (*system calls*). São cobertas as seis grandes áreas da implementação: validação segura de ponteiros de usuário, gerenciamento de processos filhos com semaforos, lock global do sistema de arquivos, proteção do executável em execução, sincronização de exec/load, e as treze syscalls implementadas.

Area	Arquivos Modificados
Validação de ponteiros de usuário	userprog/syscall.c
Gerenciamento de processos filhos	threads/thread.h, threads/thread.c, userprog/process.c
Lock global do sistema de arquivos	userprog/syscall.c, userprog/process.c
Proteção do executável (deny write)	userprog/process.c
Sincronização exec/load (load_sema)	userprog/process.c, userprog/syscall.c
Treze syscalls	userprog/syscall.c

2. ABI das System Calls

Quando um programa de usuário emite a instrução **int 0x30**, o processador muda para modo kernel e chama **syscall_handler(struct intr_frame *f)**. A pilha do usuário no momento da interrupção contém:

```
f->esp + 0 : numero da syscall (int, 32 bits)
f->esp + 4 : argumento 0
f->esp + 8 : argumento 1
f->esp + 12 : argumento 2
```

Cada slot ocupa 4 bytes. O valor de retorno da syscall deve ser escrito em **f->eax**.

Invariante de segurança: todo endereço lido do espaço do usuário deve ser validado antes de ser dereferenciado pelo kernel. Um processo malicioso pode passar qualquer padrão de bits como ponteiro. Falhar nessa verificação causa panic no kernel ou escalada de privilégios.

3. Validação de Ponteiros de Usuário

A abordagem escolhida usa **assembly inline** para capturar page faults com segurança dentro do kernel, sem usar a abordagem mais simples de verificar apenas **is_user_vaddr** e **pagedir_get_page**. Isso permite tratar ponteiros inválidos de forma atômica e eficiente.

3.1 get_user() — leitura segura de um byte

userprog/syscall.c — get_user()

```
static int32_t get_user(const uint8_t *uaddr) {
    if (!is_user_vaddr(uaddr)) /* rejeita qualquer addr >= PHYS_BASE */
        return -1;
```

```

    int result;
    asm ("movl $1f, %0; movzbl %1, %0; 1:"
        : "=&a" (result) : "m" (*uaddr));
    return result;
}

```

O funcionamento da assembly é o seguinte: **movl \$1f, %0** armazena o endereço do label **1:** em **eax**. Se o **movzbl** subsequente causar um page fault, o handler de fault verifica se o **eip** faltante corresponde ao endereço salvo e, em caso positivo, salta para **1:** — retornando -1 em **eax**. O kernel nunca crasha por ponteiro inválido de usuário.

3.2 memread_user() — leitura segura de N bytes

userprog/syscall.c — memread_user()

```

static int memread_user(const void *src, void *dst, size_t bytes) {
    for (size_t i = 0; i < bytes; i++) {
        int32_t v = get_user((const uint8_t *)src + i);
        if (v == -1) return -1;
        *(uint8_t *) (dst + i) = v & 0xff;
    }
    return (int)bytes;
}

```

Usada para ler o número da syscall e cada argumento de 32 bits da pilha do usuário para variáveis locais do kernel antes de qualquer operação.

3.3 Validação de Strings e Buffers

Strings e buffers requerem validação byte a byte porque podem cruzar fronteiras de página — a primeira página pode estar mapeada, a segunda não.

userprog/syscall.c — validate_string() e validate_buffer()

```

/* Percorre a string byte a byte até encontrar '\0'. */
static void validate_string(const char *str) {
    for (const char *p = str; ; p++) {
        if (get_user((const uint8_t *)p) == -1)
            exit_with_status(-1);
        if (*p == '\0') break;
    }
}

/* Valida cada byte de um buffer (para read/write). */
static void validate_buffer(const void *buf, unsigned size) {
    for (unsigned i = 0; i < size; i++)
        if (get_user((const uint8_t *)buf + i) == -1)
            exit_with_status(-1);
}

```

Atenção: Verificar apenas o primeiro byte de uma string é insuficiente. A string pode começar numa página válida e cruzar para uma não mapeada — o `strlen` subsequente causaria um page fault no kernel.

4. Gerenciamento de Processos Filhos

4.1 O Problema com `timer_sleep(100)`

O placeholder anterior usava `timer_sleep(100)` em `process_wait()` e sempre retornava -1. Isso tornava impossível: coletar o código de saída real do filho, detectar se o filho carregou com sucesso, e evitar que o pai encerrasse antes do filho terminar.

4.2 `struct child_process` — rastreamento por semaforo

Uma struct alocada em heap rastreia cada filho criado por `process_execute`. Dois semaforos coordenam os dois momentos criticos de sincronizacao:

- **sema** — pai bloqueia ate o filho chamar `thread_exit()` (wait).
- **load_sema** — pai bloqueia ate o filho terminar `load()` (exec).

threads/thread.h — `struct child_process`

```
struct child_process {
    tid_t tid;
    int exit_status;
    bool waited; /* impede double-wait */
    bool load_success; /* resultado de load() no filho */
    int ref_count; /* 2 = pai + filho vivos; liberado quando chega a 0 */
    struct semaphore sema; /* pai bloqueia aqui no wait */
    struct semaphore load_sema; /* pai bloqueia aqui no exec */
    struct list_elem elem;
};
```

4.3 Regras de Tempo de Vida — `ref_count`

O `ref_count` começa em 2 (pai e filho). Cada lado chama `cp_release()` exatamente uma vez. A struct e liberada por quem decrementar para zero — evitando use-after-free independente de qual lado terminar primeiro.

userprog/process.c — `cp_release()`

```
static void cp_release(struct child_process *cp) {
    enum intr_level old_level = intr_disable();
    int refs = --cp->ref_count;
    intr_set_level(old_level);
    if (refs == 0)
        free(cp);
}
```

Por que desabilitar interrupcoes? O decremento precisa ser atomico. Se pai e filho chamarem `cp_release()` simultaneamente sem protecao, ambos podem ler `ref_count=1`, decrementar para 0, e chamar `free()` duas vezes — heap corruption.

4.4 Novos Campos em struct thread

threads/thread.h — campos `USERPROG`

```
#ifdef USERPROG
    uint32_t *pagedir;
    int exit_status; /* escrito antes de thread_exit() */
#endif
```

```

    struct file *fd_table[MAX_FDS]; /* descritores abertos, indices 2..127 */
    struct list children; /* lista de child_process deste processo */
    struct child_process *cp; /* entrada do pai que rastreia esta thread */
    struct file *executable; /* executavel aberto (escrita negada) */
#endif

```

Atencao: `list_init(&t->children)` deve ser chamado explicitamente em `init_thread()`. Uma struct list zerada nao e um estado valido de lista — `list_empty()` em lista nao inicializada tem comportamento indefinido.

4.5 process_execute() — empacotando pai e filho

A decisao de usar **struct start_args** permite passar tanto a linha de comando quanto o ponteiro para cp para o filho sem alterar a assinatura de `start_process` (que recebe um unico void*).

userprog/process.c — process_execute() resumido

```

/* Aloca cp com ref=2, sema=0, load_sema=0 */
struct child_process *cp = malloc(sizeof *cp);
cp->ref_count = 2;
sema_init(&cp->sema, 0);
sema_init(&cp->load_sema, 0);
cp->waited = false;
struct start_args *args = malloc(sizeof *args);
args->cmd_line = cmd_copy;
args->cp = cp;
tid = thread_create(file_name, PRI_DEFAULT, start_process, args);
cp->tid = tid;
list_push_back(&thread_current()->children, &cp->elem);

```

4.6 process_wait() — bloqueio real

userprog/process.c — process_wait()

```

int process_wait(tid_t child_tid) {
    /* Busca o filho na lista de filhos do pai */
    struct child_process *cp = NULL;
    for (struct list_elem *e = list_begin(&cur->children); ...)
        if (c->tid == child_tid) { cp = c; break; }
    if (cp == NULL || cp->waited)
        return -1; /* nao e filho, ou ja esperado antes */
    cp->waited = true;
    sema_down(&cp->sema); /* bloqueia ate filho chamar sema_up */
    int status = cp->exit_status;
    list_remove(&cp->elem);
    cp_release(cp);
    return status;
}

```

4.7 process_exit() — notificando o pai

Antes de destruir o page directory, `process_exit()` realiza quatro acoes em ordem: fecha todos os file descriptors abertos, fecha o executavel (reabilitando escrita), notifica o pai via `sema_up`, e libera as entradas de todos os filhos.

userprog/process.c — process_exit() resumido

```

/* 1. Fecha fd_table[2..127] e o executavel */

```

```
lock_acquire(&fileSYS_lock);
for (int i = 2; i < MAX_FDS; i++)
    if (cur->fd_table[i]) { file_close(cur->fd_table[i]); cur->fd_table[i] = NULL; }
if (cur->executable) { file_close(cur->executable); cur->executable = NULL; }
lock_release(&fileSYS_lock);
/* 2. Notifica o pai */
if (cur->cp) {
    cur->cp->exit_status = cur->exit_status;
    sema_up(&cur->cp->sema);
    cp_release(cur->cp);
}
/* 3. Libera entradas de filhos */
while (!list_empty(&cur->children)) {
    struct child_process *cp = list_entry(list_pop_front(&cur->children), ...);
    cp_release(cp);
}
```

5. Lock Global do Sistema de Arquivos

Todas as funções do filesystem do PintOS (`fileysys_open`, `file_read`, `file_write`, etc.) **não são thread-safe**. Chamadas concorrentes de múltiplas threads corrompem o estado interno. A decisão é usar um único lock global que serializa todo o acesso ao filesystem.

userprog/syscall.c — lock global

```
struct lock fileysys_lock; /* declarado globalmente em syscall.c */
void syscall_init(void) {
    lock_init(&fileysys_lock);
    intr_register_int(0x30, 3, INTR_ON, syscall_handler, "syscall");
}
```

Por que um lock global e não por arquivo? O filesystem do PintOS não tem locking interno. Um lock por inode exigiria modificar os internos do filesystem. O lock grosso e correto e simples — os testes syn-read e syn-write verificam especificamente essa correção.

Atenção: `exit_with_status()` nunca deve ser chamado enquanto `fileysys_lock` está sendo mantido. Se o processo encerrar com o lock adquirido, a próxima operação de filesystem de qualquer thread entrará em **deadlock permanente**.

6. Proteção do Executável em Execução

Um processo em execução não pode permitir que seu executável seja modificado em disco enquanto roda — isso corromperia o código sendo executado. A solução é manter o arquivo aberto e negar escritas durante toda a vida do processo.

userprog/process.c — load(), no label done:

```
if (success) {
    file_deny_write(file); /* bloqueia escritas de outros processos */
    t->executable = file; /* mantido aberto; NÃO fechado aqui */
} else {
    file_close(file);
}
```

Quando `process_exit()` fecha `cur->executable`, `file_close()` chama `file_allow_write()` automaticamente — restaurando a permissão de escrita no arquivo após o processo encerrar. O lock deve envolver o `load()` em `start_process`:

userprog/process.c — start_process()

```
lock_acquire(&fileysys_lock);
success = load(executable_name, &if_.eip, &if_.esp);
lock_release(&fileysys_lock);
```

7. Sincronização exec/load com load sema

`SYS_EXEC` não pode retornar um pid ao pai antes de saber se o filho carregou com sucesso. Sem essa sincronização, o pai não consegue distinguir entre 'filho iniciou corretamente' e 'load falhou' — ambos retornariam um tid válido.

sys_exec — pai bloqueia no load_sema

```
/* Apos process_execute criar a thread filho: */  
sema_down(&cp->load_sema); /* pai bloqueia aqui */  
if (!cp->load_success)  
    return -1; /* load falhou -> exec retorna -1 */  
return (pid_t)tid;
```

start_process — filho sinaliza independente do resultado

```
cp->load_success = success;  
sema_up(&cp->load_sema); /* acorda o pai independente de sucesso ou falha */  
if (!success) thread_exit(); /* falhou: encerra o filho */
```

Atencao: *sema_up(&cp->load_sema) deve ser chamado mesmo quando load() falha. Se o filho chamar thread_exit() sem sinalizar, sys_exec bloqueia o pai para sempre — deadlock.*

8. As Treze Syscalls

Todas as syscalls são despachadas a partir de um switch em `syscall_handler()`. Cada caso lê seus argumentos via `memread_user()`, valida ponteiros e strings quando aplicável, e escreve o retorno em `f->eax`.

Syscall	Argumentos (esp+4, +8, +12)	Retorno
SYS_HALT	—	não retorna
SYS_EXIT	int status	não retorna
SYS_EXEC	char *cmdline	pid ou -1
SYS_WAIT	tid_t pid	exit status ou -1
SYS_CREATE	char *file, unsigned size	bool
SYS_REMOVE	char *file	bool
SYS_OPEN	char *file	fd ou -1
SYS_FILESIZE	int fd	bytes
SYS_READ	int fd, void *buf, unsigned size	bytes lidos
SYS_WRITE	int fd, void *buf, unsigned size	bytes escritos
SYS_SEEK	int fd, unsigned pos	void
SYS_TELL	int fd	posição atual
SYS_CLOSE	int fd	void

8.1 `exit_with_status()` — helper central

Toda syscall que precisa encerrar o processo (SYS_EXIT, acesso inválido, fd inválido) chama este helper, que garante a ordem correta das operações:

userprog/syscall.c — `exit_with_status()`

```
static void exit_with_status(int status) {
    struct thread *cur = thread_current();
    printf("%s: exit(%d)\n", cur->name, status);
    cur->exit_status = status; /* registrado ANTES de thread_exit */
    thread_exit(); /* dispara process_exit() */
}
```

8.2 Tabela de Descritores de Arquivo

Índices 0 (stdin) e 1 (stdout) são tratados diretamente nas syscalls SYS_READ e SYS_WRITE. Arquivos reais ocupam `fd_table[2..127]`. SYS_OPEN encontra o primeiro slot NULL disponível:

userprog/syscall.c — SYS_OPEN

```
struct file *opened = filesys_open(filename);
if (opened == NULL) { f->eax = -1; break; }
```

```

int fd = -1;
for (int i = 2; i < MAX_FDS; i++)
    if (cur->fd_table[i] == NULL) { cur->fd_table[i] = opened; fd = i; break; }
if (fd == -1) { /* sem slot livre */
    lock_acquire(&filesys_lock);
    file_close(opened);
    lock_release(&filesys_lock);
}
f->eax = fd;

```

8.3 SYS_READ e SYS_WRITE — stdin/stdout vs arquivo

Ambas as syscalls tratam fd=0/1 diretamente, sem passar pelo filesystem lock, e delegam para o filesystem com lock para qualquer outro fd:

userprog/syscall.c — SYS_READ

```

if (fd == 0) { /* stdin */
    uint8_t *buf = buffer;
    for (unsigned i = 0; i < size; i++) buf[i] = input_getc();
    f->eax = size;
} else {
    struct file *file = get_file_from_fd(fd);
    if (!file) { f->eax = -1; break; }
    lock_acquire(&filesys_lock);
    f->eax = file_read(file, buffer, size);
    lock_release(&filesys_lock);
}

```

8.4 SYS_EXEC — validacao e sincronizacao

SYS_EXEC valida a string da linha de comando, chama process_execute (que cria cp e a thread filho), e bloqueia em load_sema antes de retornar o pid:

userprog/syscall.c — sys_exec()

```

static pid_t sys_exec(const char *cmdline) {
    validate_string(cmdline);
    tid_t tid = process_execute(cmdline);
    if (tid == TID_ERROR) return -1;
    /* Encontra cp na lista de filhos */
    struct child_process *cp = /* busca por tid */;
    sema_down(&cp->load_sema); /* espera load() terminar */
    if (!cp->load_success) return -1;
    return (pid_t)tid;
}

```

9. Fluxo Completo exec + wait

```

Pai: sys_exec("echo hello")
|
+-- validate_string(cmdline)
+-- process_execute("echo hello")
| +-- palloc cmd_copy
| +-- malloc cp (ref=2, sema=0, load_sema=0)
| +-- malloc start_args {cmd_copy, cp}
| +-- thread_create("echo", ..., start_process, args)
| +-- list_push_back(&pai->children, &cp->elem)
|
+-- sema_down(&cp->load_sema) [PAI BLOQUEIA]
|
| Filho: start_process(args)
| +-- cur->cp = cp
| +-- lock_acquire(&fileSYS_lock)
| +-- load("echo", &eip, &esp)
| | +-- file_deney_write(file)
| | +-- t->executable = file
| +-- lock_release(&fileSYS_lock)
| +-- cp->load_success = true
| +-- sema_up(&cp->load_sema) [ACORDA PAI]
| +-- constroi pilha, salta para usuario
|
+-- (load_success == true) -> retorna pid
|
Pai: sys_wait(pid)
+-- busca cp por tid na lista children
+-- cp->waited = true
+-- sema_down(&cp->sema) [PAI BLOQUEIA]
|
| Filho: exit(0) -> SYS_EXIT
| +-- exit_with_status(0)
| +-- thread_exit() -> process_exit()
| +-- fecha fd_table e executable
| +-- cp->exit_status = 0
| +-- sema_up(&cp->sema) [ACORDA PAI]
| +-- cp_release(cp) ref: 2->1
|
+-- status = cp->exit_status (= 0)
+-- list_remove(&cp->elem)
+-- cp_release(cp) ref: 1->0 -> free(cp)
+-- retorna 0

```

10. Invariantes do Sistema

Invariante	Como e Garantido
Kernel nunca crasha por ponteiro invalido do usuario	get_user usa assembly inline com recuperacao de page fault; todas as leituras da pilha do usuario passam por memread_user.
Exit status sempre chega ao pai	process_exit armazena o status em cp e chama sema_up antes de destruir o page directory.

process_wait nao pode ser chamado duas vezes	cp->waited e verificado antes de sema_down; segunda chamada retorna -1 imediatamente.
struct child_process nao e liberada enquanto ambos os lados estao vivos	ref_count comeca em 2; cada lado chama cp_release exatamente uma vez; free so ocorre quando ref=0.
Executavel nao modificado enquanto processo roda	file_deny_write em load(); file_allow_write restaurado automaticamente por file_close em process_exit.
Filesystem thread-safe	Todas as chamadas a filesystem_* e file_* sao protegidas por lock_acquire/release(&filesystem_lock).
Sem vazamento de fd ao encerrar	process_exit fecha todos os entries nao-NULL de fd_table[2..127].
exec retorna -1 se load falhar	Pai bloqueia em load_sema; filho sinaliza com load_success=false; pai checa antes de retornar o pid.

11. Testes Relevantes

Teste	O que Valida
halt	SYS_HALT desliga a maquina.
exit	SYS_EXIT e propagacao correta do codigo de saida.
write-normal	SYS_WRITE para stdout.
create	SYS_CREATE cria arquivo.
open / close	SYS_OPEN retorna fd valido; SYS_CLOSE libera slot.
read	SYS_READ de arquivo.
filesize	SYS_FILESIZE retorna tamanho correto.
exec-once	SYS_EXEC basico — filho executa e encerra.
exec-arg	SYS_EXEC com argumentos passados corretamente.
exec-missing	exec de programa inexistente retorna -1.
wait-simple	SYS_WAIT retorna o exit code do filho.
wait-twice	Segunda chamada wait no mesmo pid retorna -1.
wait-killed	wait em filho terminado por kernel retorna -1.
syn-read	Leituras concorrentes — corretude do filesystem_lock.
syn-write	Escritas concorrentes — corretude do filesystem_lock.
rox-simple	Executavel com write-deny enquanto roda.

multi-oom	Sem vazamento de memória em múltiplos ciclos exec+wait.
-----------	---

Executar os testes

```
cd pintos/src/userprog/build
# teste individual:
pintos -p tests/userprog/exec-once -a exec-once -- -q run exec-once
# suite completa:
make check
```

Os testes syn-* podem falhar de forma intermitente se o `fileysys_lock` estiver ausente ou mal posicionado. Execute `make check` pelo menos 3 vezes para confirmar consistência.